House Price MLOps

Panduan mengajar

Panduan lengkap 23 tahap notebook: kode, penjelasan, hasil, lalu modul, DVC, Docker, dan CI/CD.

Python 3.13 / DVC / MLflow / Docker / GitHub Actions / GHCR

Repository dan sumber editable tersedia di GitHub. Salin perintah panjang dari Markdown agar baris perintah tetap utuh.

Daftar isi

Daftar isi 2

1. Masalah yang ingin diselesaikan 4

2. MLOps, MLflow, DVC, CI, dan CD 5

3. Agenda dan metode mengajar 6

4. Persiapan komputer, terminal, dan environment 7

5. Jalankan server MLflow sebelum notebook 8

6. Peta file dan kontrak input 9

7. Notebook tahap 01 - Pastikan environment dan dependency 10

8. Notebook tahap 02 - Import alat yang akan dipakai 11

9. Notebook tahap 03 - Tentukan folder kerja dan parameter umum 12

10. Notebook tahap 04 - Tulis helper untuk menjalankan perintah 13

11. Notebook tahap 05 - Inisialisasi Git dan DVC 14

12. Notebook tahap 06 - Siapkan CSV tanpa menimpa versi yang sudah ada 15

13. Notebook tahap 07 - Catat versi file dengan DVC 16

14. Notebook tahap 08 - Baca dataset dan identitas isinya 17

15. Notebook tahap 09 - Validasi sebelum training 18

16. Notebook tahap 10 - Pisahkan fitur, target, training, dan test 19

17. Notebook tahap 11 - Bangun pipeline preprocessing dan model 20

18. Notebook tahap 12 - Tentukan kandidat hyperparameter 21

19. Notebook tahap 13 - Jalankan GridSearchCV 22

20. Notebook tahap 14 - Evaluasi model dan baseline sederhana 23

21. Notebook tahap 15 - Hubungkan ke server MLflow di port 5000 24

22. Notebook tahap 16 - Log parameter, metrik, versi data, dan model 25

23. Notebook tahap 17 - Simpan model portabel dan laporan run 26

24. Notebook tahap 18 - Load kembali dari MLflow dan dari folder model 27

25. Notebook tahap 19 - Buktikan hasil prediksi tetap sama 28

26. Notebook tahap 20 - Contoh inference dan ekspor hasil 29

27. Notebook tahap 21 - Periksa run di MLflow UI 30

28. Notebook tahap 22 - Simpan dan pulihkan riwayat data 31

29. Notebook tahap 23 - Hubungkan ke struktur kode dan CI/CD 32

30. Dari notebook lengkap ke kode modular 33

31. Praktik B - Siapkan dataset dan DVC 34

32. Praktik C - Training dan MLflow 36

33. Praktik D - Load model dan prediksi 37

34. Praktik E - Tes dan pipeline DVC 38

35. Praktik F - Docker 39

36. Praktik G - GitHub Actions dan GHCR 40

37. Demonstrasi kegagalan yang terkontrol 41

38. Troubleshooting 42

39. Penilaian dan bahan diskusi 43

40. Referensi resmi 44

Panduan ini mengikuti satu proyek nyata: data California Housing, pencarian parameter Random Forest, pencatatan eksperimen, pemuatan model, versioning data, pengujian program, dan pengiriman image aplikasi ke GHCR. Repository kelas: https://github.com/bayuzen19/mlops_house_price.

Gunakan `notebooks/experiments_all_in_one.ipynb` untuk mengajar dari awal. Seluruh implementasi ada di notebook tersebut, tanpa import dari `house_price`. Setelah alurnya dipahami, gunakan `notebooks/experiments.ipynb` untuk melihat bagaimana pekerjaan yang sama dipanggil dari modul proyek. Tidak perlu menjalankan kedua notebook bersamaan.

1. Masalah yang ingin diselesaikan

Bayangkan model minggu lalu memberi hasil yang baik. Hari ini rekan tim ingin menggunakannya, tetapi tidak tahu CSV mana yang dipakai, bagaimana fitur diubah, dan apakah model perlu dilatih lagi. Notebook yang masih berjalan di laptop pembuatnya belum cukup untuk menjawab kebutuhan tersebut.

Dalam kelas ini peserta membuat bukti yang saling terhubung: versi CSV, metadata DVC, parameter, skor cross-validation, skor test, run MLflow, model portabel, hasil prediksi setelah load, commit kode, dan image aplikasi. Tujuannya agar hasil dapat ditelusuri dan dijalankan kembali oleh orang lain.

Target `MedHouseVal` mempunyai satuan 100.000 USD. Prediksi 2,5 berarti 250.000 USD pada satuan dataset historis ini. Nilai tersebut bukan estimasi harga pasar rumah masa kini. Evaluasi dilakukan pada holdout test; prediksi contoh lima baris hanya menunjukkan cara inference.

CD pada proyek ini berakhir ketika Docker image tersedia di GHCR. Workflow tidak menjalankan layanan prediksi publik. Dataset dan model hasil training diberikan saat runtime; image berisi aplikasi serta dependency yang diperlukan.

2. MLOps, MLflow, DVC, CI, dan CD

MLOps adalah cara mengelola pekerjaan machine learning supaya dapat diulang, diperiksa, dibagikan, dan dipelihara. Perubahan yang perlu dikelola mencakup kode, data, parameter, model, dan lingkungan eksekusi. Karena itu, skor model yang baik belum menjawab bagaimana model dibuat atau bagaimana menggunakannya kembali.

Contoh masalah: dua peserta memakai kode sama tetapi CSV berbeda. Angka MSE mereka tidak bisa langsung dibandingkan. Mereka perlu mencatat data, pembagian training/test, seed, parameter, dan versi library. Pada masalah lain, model sudah tersimpan tetapi scaler tertinggal di variabel notebook; proses inference di komputer lain menjadi tidak lengkap.

Konsep atau alat	Tanggung jawab pada proyek	Bukti yang diperiksa
Git	Riwayat kode, konfigurasi, metadata kecil	Commit, branch, file .dvc
DVC	Versi isi dataset dan output pipeline	Hash, cache, dvc.lock
MLflow	Catatan eksperimen dan penyimpanan model	Run ID, params, metrics, artifacts
Docker	Paket aplikasi dan dependency	Image, tag, digest
CI	Pemeriksaan otomatis ketika perubahan diajukan	Lint, tests, build
Continuous delivery	Menyiapkan dan mengirim rilis yang telah diuji	Image tersedia di GHCR
Continuous deployment	Mengaktifkan rilis otomatis di runtime tujuan	Layanan baru melayani pengguna
GitHub Actions	Pelaksana workflow CI/CD	Run, job, step, log

Pada implementasi ini, Git menjawab kode mana, DVC menjawab data mana, dan MLflow menjawab hasil eksperimen mana. Docker mengemas program, Actions menjalankan pemeriksaan/pengiriman, sedangkan GHCR menyimpan image. Ketiganya tidak menggantikan evaluasi model pada kebutuhan penggunaan yang sebenarnya.

Diskusi awal: bila MSE membaik setelah CSV berubah, apakah kita boleh menyimpulkan algoritmanya membaik? Peserta perlu menjelaskan mengapa kondisi pembanding harus dikendalikan.

3. Agenda dan metode mengajar

Materi lengkap cocok untuk dua pertemuan sekitar tiga jam, dengan waktu tambahan untuk instalasi. Demo ringkas empat jam dapat memakai mode cepat dan melewati latihan tambahan. Durasi download/build dipengaruhi jaringan dan komputer peserta.

Urutan	Kegiatan	Waktu panduan	Bukti sebelum lanjut
Pertemuan 1 awal	Konsep, setup, kernel, server	35 menit	Environment dan MLflow siap
Pertemuan 1 data	Notebook tahap 01-10	45 menit	Data, hash, split benar
Pertemuan 1 model	Tahap 11-14	35 menit	CV terpisah dari test
Pertemuan 1 tracking	Tahap 15-20	45 menit	Log dan load berhasil
Pertemuan 1 penutup	UI dan penelusuran bukti	20 menit	Run dapat ditemukan kembali
Pertemuan 2 data	Dua versi dan restore DVC	35 menit	Hash kembali ke baseline
Pertemuan 2 kode	Modul, CLI, tes, dvc repro	50 menit	Rerun dan tes dipahami
Pertemuan 2 delivery	Docker, Actions, GHCR	60 menit	Image dan commit terlacak
Pertemuan 2 penutup	Troubleshooting dan penilaian	35 menit	Bukti praktik lengkap

Pada demo pertama, baca tujuan, jalankan satu cell, bahas hasil, lalu jawab pertanyaan pemahaman. Setelah selesai, lakukan Restart Kernel dan Run All untuk memastikan tidak ada ketergantungan pada variabel tersembunyi. Server MLflow tetap berjalan di terminal terpisah selama proses tersebut.

Kode lengkap setiap tahap dicantumkan agar pengajar dapat menunjukkan baris yang sedang dijelaskan. Salin kode dari notebook atau Markdown jika baris panjang terbungkus dalam PDF. Penjelasan setelah kode membahas arti parameter, alasan pemilihan, hasil yang perlu diperiksa, dan gejala error.

4. Persiapan komputer, terminal, dan environment

Siapkan Python 3.13, Git, editor yang mendukung Jupyter, akun GitHub, dan Docker Desktop dengan Linux containers. Internet dibutuhkan saat instalasi dependency dan pengambilan dataset awal. Peserta dapat menyelesaikan bagian Python sambil menyiapkan Docker untuk sesi kedua.

Gunakan tiga terminal: Terminal A untuk praktik/tes, Terminal B untuk server MLflow, dan Terminal C untuk Git/Docker. Terminal B dibiarkan terbuka; hentikan server dengan Ctrl+C di terminal tersebut sesudah demo selesai.

Di Terminal A, dari folder induk pilihan peserta:

```
git clone https://github.com/bayuzen19/mlops_house_price.git
Set-Location mlops_house_price
py -3.13 -m venv .venv
.\.venv\Scripts\python.exe -m pip install -r requirements-dev.txt
.\.venv\Scripts\python.exe -m pip check
.\.venv\Scripts\python.exe -m house_price --help
```

Untuk latihan GitHub, gunakan fork sendiri dan ganti URL clone dengan URL fork. Pemanggilan Python .venv secara langsung memastikan instalasi dan eksekusi memakai environment sama, tanpa memerlukan aktivasi PowerShell. Jika `py` tidak tersedia tetapi `python --version` menunjukkan 3.13, gunakan `python -m venv .venv`.

Buka notebook all-in-one dan pilih kernel Python dari .venv proyek. Jika belum terdaftar, jalankan perintah berikut lalu pilih House Price (Python 3.13):

```
.\.venv\Scripts\python.exe -m ipykernel install --user --name house-price --display-name "House Price (Python 3.13)"
```

Checkpoint: pip check menyatakan dependency konsisten; CLI menampilkan prepare-data, train, predict; path kernel berada di .venv proyek. Jangan melanjutkan training jika instalasi gagal atau kernel berbeda.

Untuk ZIP kelas, ekstrak seluruh folder proyek, buka root hasil ekstraksi, lalu mulai dari pembuatan .venv. ZIP tidak memuat .git, data mentah, cache, model, atau database MLflow. Notebook all-in-one dapat menginisialisasi Git/DVC dan menyiapkan dataset publik awal. Versi data modifikasi memerlukan cache/remote yang sesuai.

5. Jalankan server MLflow sebelum notebook

Di Terminal B, masuk ke root proyek yang sama dengan Terminal A. Buat folder penyimpanan server, lalu jalankan:

```
New-Item -ItemType Directory -Force .mlflow/server
.\.venv\Scripts\python.exe -m mlflow server --host 127.0.0.1 --port 5000 --backend-store-uri
sqlite:///.mlflow/server/mlflow.db --artifacts-destination .mlflow/server/artifacts --serve-artifacts
```

Buka http://127.0.0.1:5000. Notebook mengirim run dan model ke alamat ini. Jangan menutup terminal server selama notebook masih melakukan logging atau loading model.

Bagian perintah	Arti	Dampak pada praktik
--host 127.0.0.1	Mendengarkan koneksi lokal	Diakses dari komputer yang sama
--port 5000	Port HTTP server/UI	Sama dengan tracking URI notebook
--backend-store-uri	Database metadata	Menyimpan experiments, runs, params, metrics
--artifacts-destination	Folder berkas artifact	Menyimpan file model dan metadata yang dilog
--serve-artifacts	Transfer artifact melalui server	Client memakai endpoint HTTP server

Server menampilkan UI sekaligus endpoint tracking; tidak perlu menyalakan mlflow ui tambahan untuk run ini. Database server berada di .mlflow/server, terpisah dari database SQLite mode offline lama. Run lama tidak berpindah otomatis ke server baru.

Jika port terpakai: periksa apakah yang berjalan sudah server MLflow yang dimaksud. Jika perlu port lain, ubah port server dan MLFLOW_TRACKING_URI pada environment kernel/konfigurasi notebook secara konsisten sebelum memulai run. Jangan mematikan proses yang belum diidentifikasi.

6. Peta file dan kontrak input

Notebook all-in-one menyimpan parameter langsung pada cell, termasuk QUICK_DEMO. CLI dan notebook modular membaca konfigurasi YAML. Mengubah configs/quick.yaml tidak mengubah parameter inline notebook all-in-one. Sebaliknya, mengganti QUICK_DEMO tidak mengubah CLI.

File/folder	Tanggung jawab	Cara memakainya
notebooks/experiments_all_in_one.ipynb	Implementasi lengkap dari awal	Jalankan tahap berurutan
notebooks/experiments.ipynb	Pemanggilan modul proyek	Pelajari setelah alur lengkap
src/house_price	Data, konfigurasi, train, predict, CLI	Dipakai tes dan container
params.yaml	Grid lengkap, 24 kandidat	Pipeline DVC/default CLI
configs/quick.yaml	Grid demo, dua kandidat	CLI kelas cepat
data/raw/*.csv.dvc	Identitas isi dataset	Commit ke Git
.mlflow/server	Store server lokal	Simpan untuk riwayat eksperimen
artifacts	Model portabel/laporan/prediksi	Gunakan kembali pada inference
.github/workflows/ci-cd.yml	Pemeriksaan dan publikasi otomatis	Dibaca GitHub Actions

Model menerima delapan fitur mentah: MedInc, HouseAge, AveRooms, AveBedrms, Population, AveOccup, Latitude, Longitude. MedHouseVal adalah target training/evaluasi. Pipeline tersimpan sudah mempunyai scaler; jangan melakukan fit_transform lagi pada data inference.

7. Notebook tahap 01 - Pastikan environment dan dependency

Tujuan: memasang library pada Python yang benar-benar dipakai kernel. `sys.executable` menunjukkan lokasi Python tersebut. `find_spec` memeriksa apakah modul tersedia; instalasi dilakukan hanya jika ada modul yang belum terpasang.

Daftar versi mengikuti environment proyek yang sudah diuji. Pilih kernel Python 3.13 dari `.venv`. Jika menggunakan folder proyek, instalasi melalui `requirements-dev.txt` di terminal juga sudah mencukupi.

Hasil yang diperiksa: lokasi Python menunjuk environment yang diinginkan dan tidak ada error import. Bila kernel sudah memuat versi lama lalu paket berubah, restart kernel sebelum melanjutkan.

Kode yang dijalankan

```
import importlib.util
import subprocess
import sys

packages = {
    "mlflow": "mlflow==3.17.2",
    "sklearn": "scikit-learn==1.9.1",
    "skops": "skops==0.16.0",
    "pandas": "pandas==3.1.0",
    "numpy": "numpy==2.5.4",
    "yaml": "PyYAML==6.0.3",
    "dvc": "dvc==3.67.1",
}
missing = [requirement for module, requirement in packages.items()
           if importlib.util.find_spec(module) is None]
if missing:
    subprocess.run([sys.executable, "-m", "pip", "install", *missing], check=True)
print("Python kernel:", sys.executable)
```

Penjelasan kode dan alasan

Nama import dan nama paket pip tidak selalu sama: sklearn berasal dari scikit-learn, sedangkan yaml berasal dari PyYAML. find_spec hanya memeriksa apakah modul ditemukan; versi yang sudah terpasang tidak otomatis disamakan. sys.executable menunjuk interpreter kernel, sehingga pemasangan library dilakukan pada Python yang benar. check=True menghentikan cell jika pip gagal.

Bila versi berbeda dari kebutuhan proyek, instal requirements-dev dengan kernel yang benar lalu restart.

Pemeriksaan pemahaman

Mengapa pip dari terminal global dapat berbeda dari pip kernel?

8. Notebook tahap 02 - Import alat yang akan dipakai

`pandas` menangani tabel, `numpy` membantu validasi angka, dan `scikit-learn` menyediakan split, preprocessing, model, tuning, serta metrik. MLflow menyimpan hasil eksperimen dan model. `hashlib` menghitung identitas isi data, sedangkan `yaml` membaca metadata DVC.

Import dikelompokkan berdasarkan fungsi agar peserta mudah menemukan asal setiap komponen. Semua fungsi proyek akan ditulis atau dirangkai di notebook ini.

Kode yang dijalankan

```
import hashlib
import json
import shutil
import time
from datetime import datetime
from importlib.metadata import version
from io import BytesIO
from pathlib import Path
from uuid import uuid4

import mlflow
import mlflow.sklearn
import numpy as np
import pandas as pd
import yaml
from mlflow.models import infer_signature
from sklearn.datasets import fetch_california_housing
from sklearn.dummy import DummyRegressor
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_squared_error, r2_score
from sklearn.model_selection import GridSearchCV, train_test_split
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler

for package in ["mlflow", "scikit-learn", "pandas", "numpy", "skops", "dvc"]:
    print(f"{package}: {version(package)}")
```

Penjelasan kode dan alasan

Path mengelola lokasi file, BytesIO menyediakan buffer byte untuk pandas, dan hashlib menghitung identitas isi file.

DummyRegressor menjadi pembanding sederhana sebelum menilai Random Forest. infer_signature menyimpan schema contoh input/output, bukan aturan kelayakan bisnis. version menampilkan versi library yang benar-benar dipakai; catat informasi itu sebelum membandingkan hasil antar komputer.

Pemeriksaan pemahaman

Library mana yang mengurus data versioning dan mana yang memilih model?

9. Notebook tahap 03 - Tentukan folder kerja dan parameter umum

Jika notebook dijalankan dari folder `notebooks`, root proyek adalah folder induknya. Jika notebook dipindahkan ke folder lain, folder aktif akan menjadi root. Semua file menggunakan path absolut yang diturunkan dari root ini sehingga lokasi penyimpanannya jelas.

`RANDOM_STATE` membuat pembagian data dan model dapat diulang. `TEST_SIZE=0.3` berarti 30% data dipakai sebagai test set. `QUICK_DEMO=True` memilih grid kecil untuk kelas; ubah menjadi `False` untuk grid lengkap.

Periksa: path yang dicetak adalah folder proyek yang benar sebelum menjalankan cell yang menulis file.

Kode yang dijalankan

```
current_folder = Path.cwd().resolve()
PROJECT_ROOT = current_folder.parent if current_folder.name == "notebooks" else current_folder
DATA_PATH = PROJECT_ROOT / "data" / "raw" / "california_housing.csv"
DVC_PATH = DATA_PATH.with_suffix(".csv.dvc")
DATA_RELATIVE = DATA_PATH.relative_to(PROJECT_ROOT).as_posix()

RANDOM_STATE = 42
TEST_SIZE = 0.3
QUICK_DEMO = True
FEATURES = ["MedInc", "HouseAge", "AveRooms", "AveBedrms",
            "Population", "AveOccup", "Latitude", "Longitude"]
TARGET = "MedHouseVal"

print("Root proyek:", PROJECT_ROOT)
print("Dataset:", DATA_PATH)
```

Penjelasan kode dan alasan

resolve mengubah folder menjadi path absolut.

Pemeriksaan nama notebooks mengarahkan root satu tingkat ke atas bila kernel dibuka dari sana; cocokkan print dengan lokasi proyek.

TEST_SIZE 0.3 menyisihkan 30 persen baris, sedangkan RANDOM_STATE menjaga pembagian sama pada input berurutan yang sama.

QUICK_DEMO mengendalikan grid notebook, bukan YAML.

TARGET harus dipisahkan dari FEATURES agar jawaban tidak masuk sebagai fitur.

Pemeriksaan pemahaman

Apa yang terjadi jika root proyek salah?

10. Notebook tahap 04 - Tulis helper untuk menjalankan perintah

`subprocess.run` menjalankan command sebagai daftar argumen. `cwd` memastikan Git/DVC bekerja di root proyek, bukan di folder kernel yang kebetulan aktif. `check_returncode()` menghentikan cell jika command gagal, sehingga langkah berikutnya tidak memakai hasil yang belum terbentuk.

`run_dvc` selalu memakai `sys.executable -m dvc`. Ini menghindari tertukarnya DVC dari environment Python lain. Helper juga menyediakan PATH Python kernel untuk stage DVC yang menjalankan perintah `python`.

Kode yang dijalankan

```
import os

def run_command(*arguments):
    command_env = os.environ.copy()
    command_env["PATH"] = str(Path(sys.executable).parent) + os.pathsep + command_env["PATH"]
    completed = subprocess.run(
        list(arguments), cwd=PROJECT_ROOT, env=command_env,
        stdin=subprocess.DEVNULL, capture_output=True,
        text=True, encoding="utf-8", errors="replace",
    )
    if completed.stdout.strip():
        print(completed.stdout.strip())
    if completed.stderr.strip():
        print(completed.stderr.strip())
    completed.check_returncode()
    return completed.stdout.strip()


def run_dvc(*arguments):
    return run_command(sys.executable, "-m", "dvc", *arguments)
```

Penjelasan kode dan alasan

Command dikirim sebagai list argumen, sehingga path berisi spasi tidak perlu dirangkai sebagai satu string shell. cwd memastikan DVC/Git berjalan pada proyek yang dimaksud.

Salinan environment mempertahankan pengaturan pengguna; direktori Python kernel ditambahkan ke PATH untuk subprocess. stdout dan stderr ditampilkan sebelum return code diperiksa.

Peringatan di stderr belum tentu berarti gagal; check_returncode menentukan kegagalan command.

Pemeriksaan pemahaman

Mengapa cwd penting bagi Git dan DVC?

11. Notebook tahap 05 - Inisialisasi Git dan DVC

Git menyimpan riwayat kode dan metadata; DVC menyimpan isi data pada cache serta menulis file rujukan `.dvc`. Setup hanya dijalankan bila belum ada. Pada folder di dalam repo Git yang lebih besar, `--subdir` menempatkan proyek DVC di folder ini.

Aturan ignore mencegah environment, cache, model lokal, dan database MLflow masuk Git. `.dvcignore` juga mengecualikan penyimpanan MLflow agar salinan metadata `.dvc` yang dicatat sebagai artifact tidak dianggap sebagai stage DVC baru.

Hasil: folder `.git`/`.dvc` tersedia dan versi DVC tercetak. Cell ini tidak melakukan commit atau push.

Kode yang dijalankan

```
if shutil.which("git") is None:
    raise RuntimeError("Git belum tersedia. Instal Git lalu buka ulang terminal/editor.")

probe = subprocess.run(["git", "rev-parse", "--show-toplevel"], cwd=PROJECT_ROOT,
                       capture_output=True, text=True)
if probe.returncode != 0:
    run_command("git", "init")
    git_root = PROJECT_ROOT
else:
    git_root = Path(probe.stdout.strip()).resolve()

if not (PROJECT_ROOT / ".dvc").exists():
    run_dvc("init", *(() if git_root == PROJECT_ROOT else ("--subdir",)))

for filename, rules in {
    ".gitignore": [".venv/", "__pycache__/", ".ipynb_checkpoints/", ".mlflow/",
                   "artifacts/", "mlruns/", "mlartifacts/", ".env", ".env.*"],
    ".dvcignore": [".venv/", "__pycache__/", ".mlflow/", "mlruns/", "mlartifacts/"],
}.items():
    path = PROJECT_ROOT / filename
    existing = path.read_text(encoding="utf-8") if path.exists() else ""
    missing_rules = [rule for rule in rules if rule not in existing.splitlines()]
    if missing_rules:
        with path.open("a", encoding="utf-8") as stream:
            stream.write(("\n" if existing and not existing.endswith("\n") else "")
                         + "\n".join(missing_rules) + "\n")
run_dvc("--version")
```

Penjelasan kode dan alasan

shutil.which memastikan Git terlihat pada PATH. rev-parse menemukan root repository dan git init hanya dilakukan jika probe gagal.

Jika Git root berada di induk, dvc init --subdir menempatkan DVC pada subproyek.

Ignore rules hanya ditambah jika belum ada. .gitignore mengendalikan file yang diabaikan Git; .dvcignore mencegah pemindaian DVC pada tracking artifacts, termasuk snapshot .dvc yang dilog MLflow.

Pemeriksaan pemahaman

Apa perbedaan .gitignore dan .dvcignore?

12. Notebook tahap 06 - Siapkan CSV tanpa menimpa versi yang sudah ada

Bila CSV tersedia, notebook memakai file tersebut. Bila CSV belum ada, dataset publik California Housing diambil melalui scikit-learn. Jika metadata DVC sudah ada dari clone, hasil pengambilan data harus cocok dengan hash yang tercatat sebelum file ditulis.

Dua variasi line ending diperiksa karena snapshot awal dapat berasal dari Windows. Ini tidak mengubah angka dataset; perbedaannya berada pada byte pemisah baris. Jika hash tetap berbeda, versi tersebut harus dipulihkan dari cache/remote DVC dan cell berhenti.

Hasil baseline: 20.640 baris, delapan fitur, dan satu target. Unduhan pertama memerlukan internet; scikit-learn dapat menggunakan cache lokal pada pengambilan berikutnya.

Kode yang dijalankan

```
if not DATA_PATH.exists():
    housing = fetch_california_housing(as_frame=True)
    candidates = [housing.frame.to_csv(index=False, lineterminator=ending).encode("utf-8")
                  for ending in ["\n", "\r\n"]]
    data_bytes = candidates[0]
    if DVC_PATH.exists():
        metadata = yaml.safe_load(DVC_PATH.read_text(encoding="utf-8"))
        expected = next(item["md5"] for item in metadata["outs"]
                        if item["path"] == DATA_PATH.name)
        data_bytes = next((item for item in candidates
                           if hashlib.md5(item, usedforsecurity=False).hexdigest() == expected), None)
        if data_bytes is None:
            raise RuntimeError("Sumber publik tidak cocok dengan versi DVC. Gunakan dvc pull/checkout.")
    DATA_PATH.parent.mkdir(parents=True, exist_ok=True)
    DATA_PATH.write_bytes(data_bytes)
    print("CSV berhasil disiapkan.")
else:
    print("Memakai CSV yang sudah ada; isinya tidak ditimpa.")
print(DATA_PATH)
```

Penjelasan kode dan alasan

exists melindungi CSV yang sudah ada dari penimpaan. as_frame=True memberi tabel bernama kolom.

LF dan CRLF membentuk byte berbeda walaupun tabel terlihat sama; bootstrap mencoba keduanya untuk mencocokkan metadata yang ada.

Bila sumber publik tidak cocok, notebook berhenti sebelum menulis CSV.

Ini melindungi versi modifikasi peserta agar tidak diganti data baseline diam-diam.

Pemeriksaan pemahaman

Mengapa CSV yang sudah ada tidak diunduh ulang?

13. Notebook tahap 07 - Catat versi file dengan DVC

`dvc add` menyimpan isi CSV ke cache DVC dan membuat/memperbarui file metadata. DVC juga menambahkan CSV ke `.gitignore` di folder data. Metadata berisi hash, ukuran file, dan nama data.

Jika file sudah berubah secara sengaja, menjalankan cell ini berarti mencatat versi baru sebagai versi kerja. Riwayat versinya baru tersimpan di Git setelah file `.dvc` di-commit. Jangan menyamakan `dvc add` dengan `git commit`.

Kode yang dijalankan

```
run_dvc("add", DATA_RELATIVE)
dataset_dvc_metadata = DVC_PATH.read_text(encoding="utf-8")
print("Metadata dataset:")
print(dataset_dvc_metadata)
run_dvc("status", DATA_RELATIVE + ".dvc")
```

Penjelasan kode dan alasan

dvc add menyimpan isi CSV ke cache lokal dan membuat/memperbarui rujukan metadata.

Operasi ini belum membuat commit Git dan belum mengunggah data ke remote.

Teks metadata disimpan dalam variabel agar dapat dibandingkan lagi sebelum logging. dvc status menunjukkan apakah target sesuai.

Untuk kolaborasi, metadata perlu di-commit dan isi data harus tersedia melalui cache/remote yang dibagikan.

Pemeriksaan pemahaman

Apakah dvc add sama dengan git commit?

14. Notebook tahap 08 - Baca dataset dan identitas isinya

CSV dibaca menjadi byte terlebih dahulu. SHA-256 dihitung dari byte yang sama dengan yang dibaca pandas, sehingga identitas data dan DataFrame merujuk pada isi file yang sama. Hash DVC juga diperiksa untuk mendeteksi perubahan setelah cell `dvc add`.

`df.head()` hanya menampilkan beberapa baris awal untuk inspeksi. DataFrame `df` tetap berisi seluruh baris.

Kode yang dijalankan

```
dataset_bytes = DATA_PATH.read_bytes()
dataset_sha256 = hashlib.sha256(dataset_bytes).hexdigest()
expected_md5 = next(item["md5"] for item in yaml.safe_load(dataset_dvc_metadata)["outs"]
                    if item["path"] == DATA_PATH.name)
if hashlib.md5(dataset_bytes, usedforsecurity=False).hexdigest() != expected_md5:
    raise RuntimeError("CSV berubah setelah dvc add. Tinjau dan catat versi yang benar.")

df = pd.read_csv(BytesIO(dataset_bytes))
print("Ukuran tabel:", df.shape)
print("SHA-256:", dataset_sha256)
df.head()
```

Penjelasan kode dan alasan

read_bytes mengambil snapshot isi file.

MD5 diperiksa terhadap .dvc, sedangkan SHA-256 dicatat untuk tag MLflow. pd.read_csv membaca BytesIO dari snapshot sama, sehingga data yang dipakai sesuai dengan hash yang dihitung. next memilih entri metadata yang path-nya sama dengan nama CSV.

Hash bukan indikator kualitas data; format akhir baris pun dapat mengubah hash.

Pemeriksaan pemahaman

Mengapa hash dan DataFrame harus berasal dari byte yang sama?

15. Notebook tahap 09 - Validasi sebelum training

Pemeriksaan ini memastikan fitur/target tersedia, semua kolom yang dipakai berupa angka, dan tidak ada nilai kosong atau infinity. Program berhenti lebih awal dengan pesan yang jelas bila kontrak input dilanggar.

Kita tidak menghapus baris atau mengisi missing value secara diam-diam. Perubahan pembersihan data perlu menjadi keputusan yang terlihat dan tercatat sebagai versi data baru.

Kode yang dijalankan

```
required_columns = FEATURES + [TARGET]
missing_columns = sorted(set(required_columns) - set(df.columns))
if missing_columns:
    raise ValueError(f"Kolom belum lengkap: {missing_columns}")
if df.empty:
    raise ValueError("Dataset kosong.")
if not all(pd.api.types.is_numeric_dtype(df[column]) for column in required_columns):
    raise ValueError("Fitur dan target harus numerik.")
if not np.isfinite(df[required_columns].to_numpy(dtype=float)).all():
    raise ValueError("Ada nilai kosong, NaN, atau infinity pada fitur/target.")
print("Validasi lolos.")
df[required_columns].describe().T
```

Penjelasan kode dan alasan

Selisih set menemukan kolom wajib yang hilang. df.empty mencegah tabel berheader tanpa baris lolos. is_numeric_dtype memeriksa tipe kolom; angka yang tersimpan sebagai teks perlu diperbaiki pada sumber. np.isfinite menolak NaN dan infinity. describe().T membantu melihat rentang tiap kolom, tetapi lolos pemeriksaan ini belum berarti data cocok untuk semua tujuan bisnis.

Pemeriksaan pemahaman

Apakah schema valid berarti data layak dipakai untuk semua bisnis?

16. Notebook tahap 10 - Pisahkan fitur, target, training, dan test

`X` berisi delapan fitur, sedangkan `y` berisi target yang ingin diprediksi. Target tidak ikut masuk ke fitur. `train_test_split` menyisihkan 30% data untuk evaluasi akhir dan memakai seed 42 agar pembagian dapat diulang.

Test set tidak digunakan untuk memilih hyperparameter. Pada baseline 20.640 baris, training berisi 14.448 baris dan test berisi 6.192 baris. Data pada tahap ini masih mentah, belum di-scaling.

Kode yang dijalankan

```
X = df.loc[:, FEATURES].copy()
y = df[TARGET].copy()
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=TEST_SIZE, random_state=RANDOM_STATE,
)
assert set(X_train.index).isdisjoint(set(X_test.index))
print("Training:", X_train.shape, "Test:", X_test.shape)
X_train.head()
```

Penjelasan kode dan alasan

X berisi fitur dan y berisi target. copy membuat objek hasil pemilihan eksplisit. train_test_split menghasilkan 14.448 baris training dan 6.192 test pada baseline. isdisjoint membuktikan indeks tidak beririsan; ini tidak mendeteksi seluruh bentuk kebocoran seperti duplikat bermakna yang berbeda indeks.

Test set tidak diberikan ke GridSearchCV.fit.

Pemeriksaan pemahaman

Mengapa test set tidak masuk grid_search.fit?

17. Notebook tahap 11 - Bangun pipeline preprocessing dan model

Pipeline merangkai `StandardScaler` dan `RandomForestRegressor`. Saat pipeline di-fit, scaler belajar dari data training, lalu hasil transformasinya masuk ke model. Saat `predict`, pipeline menggunakan statistik scaler yang sudah dipelajari.

Menaruh scaler di dalam pipeline juga membuat GridSearchCV melatih preprocessing hanya pada bagian training setiap fold. Ini menghindari kebocoran informasi dari validation fold. Random Forest biasanya tidak memerlukan scaling, tetapi scaler dipertahankan untuk memperjelas pola penyimpanan preprocessing bersama model.

`random_state` mengontrol pengacakan model. `n_jobs=1` pada Random Forest mencegah parallelism bertingkat ketika GridSearchCV nanti menjalankan fitting.

Kode yang dijalankan

```
pipeline = Pipeline([
    ("scaler", StandardScaler()),
    ("model", RandomForestRegressor(random_state=RANDOM_STATE, n_jobs=1)),
])
pipeline
```

Penjelasan kode dan alasan

Nama scaler dan model menjadi identitas tahap pipeline. fit mempelajari statistik scaler hanya dari data training lalu melatih model; predict memakai transform dengan statistik yang tersimpan.

Random Forest umumnya tidak membutuhkan scaling, tetapi contoh mempertahankannya untuk mengajarkan penyimpanan preprocessing. n_jobs=1 menjaga jumlah worker terkontrol pada demo.

Pipeline yang sama menjadi unit logging dan loading.

Pemeriksaan pemahaman

Apa yang hilang jika hanya Random Forest yang disimpan?

18. Notebook tahap 12 - Tentukan kandidat hyperparameter

Awalan `model__` mengarah ke tahap bernama `model` di dalam pipeline. Mode cepat mempunyai dua kandidat dan dua fold, sehingga empat fitting cross-validation dilakukan. Grid penuh mempunyai 24 kandidat dan tiga fold, sehingga 72 fitting dilakukan, ditambah refit model pemenang.

- `n_estimators`: jumlah pohon.

- `max_depth`: batas kedalaman pohon; `None` tidak memberi batas ini.

- `min_samples_split`: minimal sampel agar node dapat dibagi.

- `min_samples_leaf`: minimal sampel pada leaf.

`neg_mean_squared_error` dipakai karena GridSearchCV selalu memilih skor yang lebih besar. Tanda negatif dibalik saat menampilkan CV MSE.

Kode yang dijalankan

```
if QUICK_DEMO:
    param_grid = {"model__n_estimators": [20], "model__max_depth": [5, 10],
                  "model__min_samples_split": [2], "model__min_samples_leaf": [1]}
    CV_FOLDS = 2
else:
    param_grid = {"model__n_estimators": [100, 200], "model__max_depth": [5, 10, None],
                  "model__min_samples_split": [2, 5], "model__min_samples_leaf": [1, 2]}
    CV_FOLDS = 3

candidate_count = int(np.prod([len(values) for values in param_grid.values()]))
print("Kandidat:", candidate_count, "Fold:", CV_FOLDS,
      "Total fitting CV:", candidate_count * CV_FOLDS)
```

Penjelasan kode dan alasan

n_estimators menentukan jumlah pohon. max_depth membatasi kedalaman, dengan None tanpa batas kedalaman tersebut. min_samples_split membatasi jumlah sampel minimum untuk membagi node; min_samples_leaf membatasi ukuran leaf.

Mode cepat mempunyai 1x2x1x1=2 kandidat, dua fold, sehingga empat fitting CV.

Mode lengkap mempunyai 24 kandidat, tiga fold, sehingga 72 fitting CV.

Refit pemenang menambah satu fitting.

Pemeriksaan pemahaman

Berapa fitting CV pada mode lengkap dan mengapa ada refit?

19. Notebook tahap 13 - Jalankan GridSearchCV

`fit` menerima training set saja. GridSearchCV membagi training set menjadi fold, mencoba semua kandidat, menghitung skor rata-rata, lalu melatih kembali pipeline dengan parameter pemenang pada seluruh training set.

Hasil terbaik di sini berarti terbaik di antara kandidat dan skema cross-validation yang digunakan. Ini belum merupakan keputusan otomatis bahwa model siap untuk semua penggunaan bisnis.

Durasi fitting dicatat terpisah. Cell logging MLflow diletakkan setelah training agar error penyimpanan tidak memaksa peserta mengulang seluruh tuning.

Kode yang dijalankan

```
grid_search = GridSearchCV(
    estimator=pipeline, param_grid=param_grid, cv=CV_FOLDS,
    scoring="neg_mean_squared_error", n_jobs=1, verbose=1, error_score="raise",
)
started = time.perf_counter()
grid_search.fit(X_train, y_train)
training_seconds = time.perf_counter() - started
best_model = grid_search.best_estimator_
best_params = {key.removeprefix("model__"): value
               for key, value in grid_search.best_params_.items()}
print("Parameter terbaik:", best_params)
print("CV MSE:", -grid_search.best_score_)
print(f"Durasi training: {training_seconds:.2f} detik")
```

Penjelasan kode dan alasan

GridSearchCV memaksimalkan skor; MSE dibalik tanda menjadi neg_mean_squared_error agar kandidat dengan error terkecil menang. best_estimator_ adalah pipeline pemenang yang direfit pada seluruh X_train, bukan sekadar konfigurasi. error_score=raise membuat kegagalan terlihat sebagai exception. perf_counter mengukur interval training; perbedaan durasi antar komputer tidak otomatis menunjukkan bug.

Tidak ada data X_test di proses pemilihan ini.

Pemeriksaan pemahaman

Apa definisi terbaik pada best_estimator_?

20. Notebook tahap 14 - Evaluasi model dan baseline sederhana

Prediksi dibuat pada test set mentah; pipeline melakukan scaling secara otomatis. MSE menghitung rata-rata kesalahan kuadrat, RMSE mengembalikan skala ke satuan target, dan R^2 membandingkan performa dengan prediksi rata-rata. Nilai R^2 dapat negatif.

`DummyRegressor(strategy="mean")` memprediksi rata-rata target training untuk setiap sampel. Baseline ini membantu peserta menilai apakah model kompleks memberikan manfaat dibanding tebakan sederhana. Model dan baseline dinilai pada test set yang sama.

Kode yang dijalankan

```
y_pred = best_model.predict(X_test)
baseline = DummyRegressor(strategy="mean").fit(X_train, y_train)
baseline_pred = baseline.predict(X_test)
mse = float(mean_squared_error(y_test, y_pred))
metrics = {
    "mse": mse,
    "rmse": float(np.sqrt(mse)),
    "r2_score": float(r2_score(y_test, y_pred)),
    "cv_mse": float(-grid_search.best_score_),
    "baseline_mse": float(mean_squared_error(y_test, baseline_pred)),
    "training_seconds": training_seconds,
}
pd.Series(metrics, name="nilai")
```

Penjelasan kode dan alasan

Baseline mean memprediksi rata-rata target training dan memberi konteks untuk skor model.

MSE adalah rata-rata kuadrat kesalahan, RMSE akar MSE dengan satuan target, dan R2 bisa negatif.

CV MSE dihitung pada fold training, test MSE pada holdout.

Jangan mengulang pencarian parameter berdasarkan test set berkali-kali karena test kemudian ikut memengaruhi pilihan model.

Untuk membaca satuan USD dataset, RMSE dikalikan 100.000.

Pemeriksaan pemahaman

Apa arti model lebih buruk daripada baseline?

21. Notebook tahap 15 - Hubungkan ke server MLflow di port 5000

Notebook memakai `http://127.0.0.1:5000`. Sebelum menjalankan notebook, buka terminal PowerShell kedua **di root proyek** dan jalankan:

```
New-Item -ItemType Directory -Force .mlflow/server
.\.venv\Scripts\python.exe -m mlflow server --host 127.0.0.1 --port 5000 --backend-store-uri
sqlite:///.mlflow/server/mlflow.db --artifacts-destination .mlflow/server/artifacts --serve-artifacts
```

Biarkan terminal server terbuka. Buka `http://127.0.0.1:5000` untuk melihat UI. Server menerima catatan run dan file model; SQLite di belakang server menyimpan metadata, sedangkan folder artifacts menyimpan file. Notebook mengakses server melalui HTTP.

Cell berikut memeriksa koneksi dengan batas waktu singkat. Jika server belum aktif, jalankan perintah di atas lalu ulangi cell ini. Pengaturan tracking dilakukan sebelum `start_run`. Environment variable `MLFLOW_TRACKING_URI` hanya diperlukan untuk mengganti alamat secara sengaja, misalnya pada pengujian otomatis.

Kode yang dijalankan

```
from urllib.error import URLError
from urllib.request import urlopen

TRACKING_URI = os.environ.get("MLFLOW_TRACKING_URI", "http://127.0.0.1:5000")
EXPERIMENT_NAME = "house-price-all-in-one"
if TRACKING_URI.startswith(("http://", "https://")):
    try:
        with urlopen(TRACKING_URI.rstrip("/") + "/health", timeout=5) as response:
            if response.status != 200:
                raise RuntimeError("Server MLflow belum siap.")
    except (URLError, TimeoutError) as error:
        raise RuntimeError(
            f"MLflow belum dapat diakses di {TRACKING_URI}. "
            "Jalankan server sesuai petunjuk Markdown, lalu ulangi cell ini."
        ) from error

mlflow.set_tracking_uri(TRACKING_URI)
if mlflow.get_experiment_by_name(EXPERIMENT_NAME) is None:
    artifact_location = None
    if TRACKING_URI.startswith("sqlite:///"):
        db_path = Path(TRACKING_URI.removeprefix("sqlite:///"))
        artifact_location = (db_path.parent / "artifacts").resolve().as_uri()
    mlflow.create_experiment(EXPERIMENT_NAME, artifact_location=artifact_location)
mlflow.set_experiment(EXPERIMENT_NAME)
print("Tracking URI:", TRACKING_URI)
print("Experiment:", EXPERIMENT_NAME)
```

Penjelasan kode dan alasan

Tracking URI adalah alamat server yang menerima catatan dan artifact.

Health check menangkap server yang belum aktif sebelum logging, bukan menguji kualitas model. set_tracking_uri dan set_experiment harus dilakukan sebelum start_run.

Client HTTP tidak menetapkan artifact_location file lokal; lokasi artifact dikelola server.

Environment override berguna untuk pengujian offline, tetapi demo kelas memakai http://127.0.0.1:5000.

Pemeriksaan pemahaman

Di mana server menyimpan metadata dan di mana file model?

22. Notebook tahap 16 - Log parameter, metrik, versi data, dan model

Sebelum logging, checksum file dan metadata diperiksa kembali agar tidak mengaitkan model dengan versi data yang sudah berubah. Parameter menjelaskan konfigurasi; metrik menjelaskan hasil; tag dan artifact menghubungkan run ke dataset.

`infer_signature` mencatat bentuk input/output model. `input_example` memberikan contoh fitur mentah. Model disimpan memakai `skops`. Tipe `sklearn.tree._tree.Tree` dipercaya secara eksplisit karena pipeline ini dilatih sendiri di atas; kita tidak memasukkan semua tipe tidak dikenal secara otomatis.

`model_info.model_uri` menunjuk model yang benar-benar tersimpan pada run ini. Simpan URI tersebut, bukan menebak path artifact.

Kode yang dijalankan

```
if hashlib.sha256(DATA_PATH.read_bytes()).hexdigest() != dataset_sha256:
    raise RuntimeError("Dataset berubah setelah training. Ulangi proses dengan versi yang benar.")
if DVC_PATH.read_text(encoding="utf-8") != dataset_dvc_metadata:
    raise RuntimeError("Metadata DVC berubah setelah training.")

signature = infer_signature(X_train, best_model.predict(X_train))
model_options = {
    "signature": signature,
    "input_example": X_train.head(3),
    "serialization_format": "skops",
    "skops_trusted_types": ["sklearn.tree._tree.Tree"],
    "pip_requirements": [f"{name}=={version(name)}"
                         for name in ["mlflow", "scikit-learn", "skops", "pandas", "numpy"]],
}
with mlflow.start_run() as run:
    mlflow.log_params({**best_params, "random_state": RANDOM_STATE,
                       "test_size": TEST_SIZE, "cv": CV_FOLDS})
    mlflow.log_metrics(metrics)
    mlflow.set_tags({"dataset.sha256": dataset_sha256, "dataset.versioning": "dvc"})
    mlflow.log_text(dataset_dvc_metadata, "data_version/california_housing.csv.dvc")
    model_info = mlflow.sklearn.log_model(
        best_model, name="model", registered_model_name="Best Random Forest Model",
        **model_options,
    )
    RUN_ID = run.info.run_id
print("Run ID:", RUN_ID)
print("Model URI:", model_info.model_uri)
```

Penjelasan kode dan alasan

Recheck hash dan teks metadata menangkap perubahan file di tengah training.

Signature mencatat schema, input_example memberi tiga contoh fitur mentah.

Serializer skops diberi tipe sklearn.tree._tree.Tree secara spesifik karena Random Forest yang dilatih sendiri memerlukannya; ini bukan izin mempercayai semua tipe. with start_run menutup run saat blok selesai.

Params menyimpan konfigurasi, metrics menyimpan angka, tags memberi label, dan log_text menyimpan snapshot metadata.

Gunakan model_info.model_uri dari hasil API, bukan menebak versi berdasarkan nama model.

Pemeriksaan pemahaman

Mengapa checksum perlu dilog pada run yang sama?

23. Notebook tahap 17 - Simpan model portabel dan laporan run

MLflow sudah menyimpan model pada tracking store. Salinan lokal berikut memudahkan pemakaian tanpa harus mengakses tracking server. Nama folder mengandung waktu dan kode acak pendek sehingga menjalankan ulang cell tidak menimpa hasil sebelumnya.

`run.json` mencatat lokasi model, tracking URI, checksum dataset, parameter, dan metrik. Setelah kernel restart, laporan ini menjadi titik masuk untuk menemukan model kembali. Pada konfigurasi server, gunakan URL tanpa kredensial di dalam URI; autentikasi sebaiknya melalui environment yang sesuai.

Kode yang dijalankan

```
run_label = datetime.now().strftime("%Y%m%d-%H%M%S") + "-" + uuid4().hex[:6]
OUTPUT_DIR = PROJECT_ROOT / "artifacts" / "all_in_one" / run_label
OUTPUT_DIR.mkdir(parents=True)
mlflow.sklearn.save_model(best_model, path=str(OUTPUT_DIR / "model"), **model_options)
run_report = {
    "run_id": RUN_ID, "model_uri": model_info.model_uri, "tracking_uri": TRACKING_URI,
    "local_model_path": str(OUTPUT_DIR / "model"), "dataset_sha256": dataset_sha256,
    "best_params": best_params, "metrics": metrics,
    "random_state": RANDOM_STATE, "test_size": TEST_SIZE,
}
RUN_REPORT_PATH = OUTPUT_DIR / "run.json"
RUN_REPORT_PATH.write_text(json.dumps(run_report, indent=2), encoding="utf-8")
print("Simpan path ini untuk load setelah restart kernel:")
print(RUN_REPORT_PATH)
```

Penjelasan kode dan alasan

Timestamp dan UUID memberi folder unik untuk setiap hasil sehingga eksperimen sebelumnya tetap tersedia. log_model menyimpan melalui tracking MLflow, sedangkan save_model menghasilkan folder model portabel. run.json menghubungkan run ID, model URI, tracking URI, checksum, metrik, dan parameter.

Path absolut lokal mungkin berubah jika folder dipindahkan; pada mesin baru gunakan lokasi ekspor yang tersedia atau server bersama yang bisa diakses.

Pemeriksaan pemahaman

Informasi apa yang diperlukan setelah restart kernel?

24. Notebook tahap 18 - Load kembali dari MLflow dan dari folder model

Cell ini membaca laporan run dari disk, mengatur tracking URI yang benar, lalu memuat model. Setelah restart kernel, import `json`, `Path`, dan `mlflow.sklearn`, kemudian isi `RUN_REPORT_PATH` dengan path laporan yang sudah dicetak. Cell logging/training tidak perlu dijalankan lagi hanya untuk inference.

Dua objek dimuat: satu melalui URI MLflow, satu melalui folder model portabel. Keduanya mencakup scaler dan Random Forest.

Kode yang dijalankan

```
saved_run = json.loads(Path(RUN_REPORT_PATH).read_text(encoding="utf-8"))
mlflow.set_tracking_uri(saved_run["tracking_uri"])
loaded_best_model = mlflow.sklearn.load_model(saved_run["model_uri"])
loaded_local_model = mlflow.sklearn.load_model(saved_run["local_model_path"])
print("Model dari MLflow:", saved_run["model_uri"])
print("Model lokal:", saved_run["local_model_path"])
```

Penjelasan kode dan alasan

json.loads mengambil laporan dari disk.

Set tracking URI ke server/store yang menghasilkan model URI, lalu load model dari URI tersebut.

Jalur kedua memuat folder ekspor tanpa mencari experiment.

Proses ini memuat pemenang pada run tersimpan, bukan mencari model terbaik dari seluruh riwayat.

Setelah restart kernel, isi RUN_REPORT_PATH dengan path laporan yang telah dicatat sebelum menjalankan cell ini.

Pemeriksaan pemahaman

Kapan lebih mudah memuat folder model portabel?

25. Notebook tahap 19 - Buktikan hasil prediksi tetap sama

Pengujian ini membandingkan prediksi sebelum penyimpanan, sesudah load dari MLflow, dan sesudah load dari folder lokal pada input yang sama. `assert_allclose` akan menghentikan cell bila ada perbedaan di luar toleransi numerik.

Tabel aktual versus prediksi membantu inspeksi. Pemeriksaan kesamaan hasil membuktikan proses simpan/load bekerja; itu berbeda dari pembuktian kualitas model untuk kasus penggunaan baru.

Kode yang dijalankan

```
y_pred_loaded = loaded_best_model.predict(X_test)
y_pred_local = loaded_local_model.predict(X_test)
np.testing.assert_allclose(y_pred, y_pred_loaded, rtol=1e-12, atol=1e-12)
np.testing.assert_allclose(y_pred, y_pred_local, rtol=1e-12, atol=1e-12)
print("PASS: prediksi sebelum dan sesudah load sama.")
print("MSE setelah load:", mean_squared_error(y_test, y_pred_loaded))
pd.DataFrame({"Aktual": y_test, "Prediksi": y_pred_loaded}).head(10)
```

Penjelasan kode dan alasan

Tidak ada fit ulang pada tahap verifikasi.

X_test sama diberikan ke kedua hasil load. assert_allclose membandingkan setiap elemen prediksi dengan toleransi sangat kecil dan menghentikan cell jika berbeda.

MSE dihitung lagi untuk menghubungkan bukti serialisasi dengan perilaku inference.

Tabel sepuluh contoh membantu pembacaan, tetapi evaluasi keseluruhan tetap memakai semua test set.

Pemeriksaan pemahaman

Mengapa berhasil load saja belum cukup membuktikan perilaku model?

26. Notebook tahap 20 - Contoh inference dan ekspor hasil

Contoh memakai lima baris test set sebagai simulasi input baru. Fitur tetap mentah dan disusun sesuai urutan yang disimpan model. Target tidak diberikan kepada model.

Pipeline mengerjakan preprocessing secara otomatis. Hasil disimpan di folder run sebagai CSV. Angka prediksi masih dalam satuan target dataset; kolom USD di bawah hanya mengalikan satuannya dengan 100.000.

Kode yang dijalankan

```
sample_input = X_test.head(5).loc[:, FEATURES].copy()
if not np.isfinite(sample_input.to_numpy(dtype=float)).all():
    raise ValueError("Input prediksi tidak valid.")
sample_predictions = loaded_best_model.predict(sample_input)
prediction_table = pd.DataFrame({
    "prediction": sample_predictions,
    "prediction_usd_dataset_unit": sample_predictions * 100_000,
}, index=sample_input.index)
prediction_table.to_csv(OUTPUT_DIR / "sample_predictions.csv", index=False)
print("Hasil:", OUTPUT_DIR / "sample_predictions.csv")
prediction_table
```

Penjelasan kode dan alasan

head(5) sengaja membuat contoh mudah diperiksa, bukan test set baru untuk memilih model.

Kolom fitur dipilih dalam urutan yang ditentukan dan target tidak ikut.

Perkalian 100.000 hanya konversi satuan dataset, tanpa koreksi inflasi atau kondisi pasar. index=False menghindari tambahan kolom indeks pandas pada CSV hasil.

Untuk input baru, ganti sample_input dengan DataFrame yang memenuhi schema sama.

Pemeriksaan pemahaman

Bagaimana menyiapkan seratus baris input baru?

27. Notebook tahap 21 - Periksa run di MLflow UI

Buka `http://127.0.0.1:5000` di browser. Server yang dijalankan sebelum training menyediakan UI dan API tracking pada alamat yang sama.

Pilih experiment `house-price-all-in-one`, lalu periksa parameter, metrik, tag `dataset.sha256`, metadata `.dvc`, dan model. Cocokkan run ID dengan `RUN_ID` di notebook. `loaded_best_model` dimuat dari model URI pada server yang sama.

Terminal server dibiarkan berjalan selama logging atau load dari MLflow. Hentikan dengan Ctrl+C setelah selesai. Folder `.mlflow/server` menyimpan metadata dan artifacts agar run tetap tersedia saat server dinyalakan lagi dengan perintah yang sama. Run lama dari database SQLite lain tidak otomatis digabungkan ke server ini.

Penjelasan kode dan alasan

Server MLflow sudah menyediakan UI pada alamat tracking yang sama, sehingga tidak perlu proses UI kedua.

Pilih experiment yang sesuai notebook, lalu buka satu run.

Cocokkan params, metrics, tag checksum, artifact metadata DVC, dan model URI pada run tersebut.

Jika UI kosong, periksa alamat server dan nama experiment sebelum melatih ulang; run mungkin berada pada database/store berbeda.

Pemeriksaan pemahaman

Mengapa UI kosong walaupun logging sebelumnya berhasil?

28. Notebook tahap 22 - Simpan dan pulihkan riwayat data

Perintah berikut dijalankan secara sengaja di terminal. `dvc add` pada cell sebelumnya sudah menyimpan isi CSV ke cache; Git menyimpan rujukan versinya.

```
git add .dvc .dvcignore .gitignore data/raw/.gitignore data/raw/california_housing.csv.dvc
notebooks/experiments_all_in_one.ipynb
git commit -m "Record all-in-one notebook and dataset version"
git tag data-all-in-one-v1
```

Tag dibuat sekali. Setelah ada perubahan data yang disengaja, jalankan kembali cell DVC dan pembacaan data, lalu commit metadata versi baru. Untuk memulihkan baseline setelah perubahan penting sudah disimpan:

```
git restore --source=data-all-in-one-v1 -- data/raw/california_housing.csv.dvc
.\.venv\Scripts\python.exe -m dvc checkout data/raw/california_housing.csv.dvc
```

Setelah checkout, baca ulang data dan ulangi split/training jika ingin membuat model untuk versi tersebut. Variabel yang sudah berada di memori tidak berubah otomatis. Untuk berbagi versi modifikasi ke komputer lain, siapkan remote DVC lalu `dvc push`; file Git saja tidak berisi CSV.

Penjelasan kode dan alasan

Git menyimpan rujukan metadata, DVC menyimpan isi data.

Restore memerlukan dua langkah: pilih metadata dari commit/tag, lalu checkout objek DVC.

Tag Git tidak membawa CSV jika cache/remote tidak tersedia. dvc push mengirim cache ke remote, dvc pull mengambil dan memulihkan objek.

Folder remote lokal hanya simulasi; akses lintas mesin memerlukan remote yang benar-benar dapat dijangkau.

Pemeriksaan pemahaman

Mengapa clone Git belum tentu memulihkan CSV?

29. Notebook tahap 23 - Hubungkan ke struktur kode dan CI/CD

Notebook ini tetap lengkap tanpa modul proyek. Versi modular memindahkan tanggung jawab yang sama ke `data.py`, `config.py`, `train.py`, `predict.py`, dan `cli.py`. Ini memudahkan tes, CLI, serta container, sementara notebook all-in-one membantu memahami proses dari awal.

Workflow GitHub Actions mempunyai tiga tahap: `quality` memeriksa Python/tes, `container` membangun dan menguji image, dan `publish` mengirim image yang lolos ke GHCR. Push ke branch utama/tag rilis dapat memicu publikasi; pull request menjalankan pemeriksaan. Docker image berisi aplikasi modular, sedangkan model notebook tetap dapat digunakan sebagai folder model MLflow yang portabel.

Panduan detail setiap blok kode, perintah Docker, GitHub Actions, latihan DVC, dan troubleshooting tersedia pada panduan step-by-step terpisah. Gunakan notebook modular dan all-in-one sebagai dua pendekatan belajar, bukan menjalankan keduanya bersamaan dengan asumsi variabelnya saling berbagi.

Penjelasan kode dan alasan

Pemindahan kode ke modul memisahkan tanggung jawab data, konfigurasi, training, dan inference.

Tes mempertahankan perilaku input-output tersebut ketika implementasi berubah.

CI memakai data sintetis kecil agar cepat dan tidak bergantung download.

Image memuat aplikasi, bukan hasil training peserta.

CD mempublikasikan setelah kondisi job/event terpenuhi; pengaktifan layanan di server merupakan pekerjaan lanjutan.

Pemeriksaan pemahaman

Apa beda CI lolos, image tersedia, dan layanan aktif?

30. Dari notebook lengkap ke kode modular

Langkah peserta

Fork repository pengajar melalui GitHub. Peserta yang hanya mengikuti demo lokal boleh clone repository pengajar langsung. Untuk latihan commit, pull request, dan GHCR, gunakan fork pribadi agar setiap peserta mengelola repository sendiri.

Di **Terminal A**, pilih folder induk, lalu jalankan:

```
git clone https://github.com/bayuzen19/mlops_house_price.git
Set-Location mlops_house_price
py -3.13 -m venv .venv
.\.venv\Scripts\python.exe -m pip install -r requirements-dev.txt
.\.venv\Scripts\python.exe -m pip check
.\.venv\Scripts\python.exe -m house_price --help
```

Untuk fork, ganti URL clone dengan URL fork peserta. Bila `py` tidak tersedia tetapi `python --version` sudah 3.13, gunakan `python -m venv .venv`.

Hasil yang diharapkan: `pip check` melaporkan tidak ada dependency yang rusak; CLI menampilkan `prepare-data`, `train`, dan `predict`. Perintah tidak memerlukan aktivasi environment karena Python `.venv` dipanggil langsung.

Jelaskan struktur kode

File	Pertanyaan yang dijawab
`data.py`	Apakah data valid dan cocok dengan versi DVC?
`config.py`	Parameter apa yang digunakan?
`train.py`	Bagaimana training, evaluasi, dan penyimpanan dijalankan?
`predict.py`	Bagaimana model digunakan kembali?
`versioning.py`	Bagaimana notebook memanggil DVC?
`cli.py`	Bagaimana semua fungsi dijalankan lewat terminal?
`tests/`	Perilaku apa yang harus tetap benar setelah perubahan?
`notebooks/experiments.ipynb`	Bagaimana alur tersebut dijelaskan secara bertahap?

Buka notebook modular `notebooks/experiments.ipynb`. Tunjukkan satu fungsi di notebook dan cari fungsi yang sama di `src/house_price`. Tekankan bahwa notebook menggunakan modul yang sama dengan CLI dan container.

31. Praktik B - Siapkan dataset dan DVC

Di **Terminal A**:

```
.\.venv\Scripts\python.exe -m house_price prepare-data --from-source
.\.venv\Scripts\python.exe -m dvc add data/raw/california_housing.csv
.\.venv\Scripts\python.exe -m dvc status data/raw/california_housing.csv.dvc
Get-Content data/raw/california_housing.csv.dvc
git status --short
```

Bootstrap `--from-source` hanya cocok untuk dataset publik awal. Program membandingkan hasil pengambilan data dengan hash metadata DVC. Jika versinya berbeda, program berhenti tanpa membuat CSV. Untuk versi data yang telah dimodifikasi, gunakan cache atau remote DVC yang menyimpan versi tersebut.

Hasil yang diharapkan: CSV memiliki 20.640 baris dan 9 kolom termasuk target; DVC melaporkan data sesuai metadata. File `.dvc` memuat `md5`, `size`, dan `path`. CSV tidak ditambahkan ke Git.

Periksa ukuran data:

```
.\.venv\Scripts\python.exe -c "from house_price.data import load_dataset;
d=load_dataset('data/raw/california_housing.csv'); print(d.frame.shape); print(d.sha256)"
```

Jelaskan dua checksum yang terlihat: DVC menggunakan MD5 sebagai identitas isi file pada metadata ini; aplikasi mencatat SHA-256 ke MLflow. Keduanya dihitung dari byte CSV yang dipakai. Hash bukan metrik kualitas model.

Eksperimen versi data

Lakukan pada fork/clone latihan milik peserta. Pastikan perubahan lain sudah disimpan dan baseline dataset sudah masuk cache melalui `dvc add`.

```
git tag data-baseline
.\.venv\Scripts\python.exe -c "import pandas as pd; p='data/raw/california_housing.csv'; d=pd.read_csv(p);
d.sample(frac=0.8,random_state=42).to_csv(p,index=False)"
.\.venv\Scripts\python.exe -m dvc status data/raw/california_housing.csv.dvc
```

Hasil yang diharapkan: DVC mendeteksi perubahan. Dataset latihan kini memiliki 16.512 baris. Perubahan ini sengaja dibuat untuk mempelajari versioning; belum berarti kualitas model membaik.

Coba training sebelum `dvc add`. Program seharusnya menolak CSV yang tidak cocok dengan metadata. Lalu catat versi baru:

```
.\.venv\Scripts\python.exe -m dvc add data/raw/california_housing.csv
git add data/raw/california_housing.csv.dvc data/raw/.gitignore
git commit -m "Track 80 percent classroom data variant"
git log --oneline -- data/raw/california_housing.csv.dvc
```

Jika Git meminta identitas, peserta mengisi nama dan email mereka sendiri melalui `git config user.name` dan `git config user.email`, kemudian mengulangi commit.

Pulihkan baseline:

```
git restore --source=data-baseline -- data/raw/california_housing.csv.dvc
.\.venv\Scripts\python.exe -m dvc checkout data/raw/california_housing.csv.dvc
.\.venv\Scripts\python.exe -c "from house_price.data import load_dataset;
d=load_dataset('data/raw/california_housing.csv'); print(d.frame.shape); print(d.sha256)"
```

Bukti: jumlah baris kembali 20.640 dan checksum sama dengan baseline. Metadata Git pada working tree kini menunjuk baseline. Untuk mempertahankan baseline sebagai commit terbaru:

```
git add data/raw/california_housing.csv.dvc
git commit -m "Restore baseline dataset for model lab"
```

Jangan menghapus cache DVC selama latihan restore. Git menyimpan rujukan versi; isi CSV perlu tersedia di cache atau remote agar dapat dipulihkan.

Opsional: remote DVC lokal

Untuk memahami `push` dan `pull`, gunakan folder backup di luar repo. Contoh berikut memakai folder di profil pengguna; ini hanya simulasi penyimpanan terpisah pada komputer yang sama.

```
$classBackup = Join-Path $env:USERPROFILE 'dvc-classroom-backup'
New-Item -ItemType Directory -Force -Path $classBackup
.\.venv\Scripts\python.exe -m dvc remote add --local -d classroom $classBackup
.\.venv\Scripts\python.exe -m dvc push data/raw/california_housing.csv.dvc
.\.venv\Scripts\python.exe -m dvc pull data/raw/california_housing.csv.dvc
```

`--local` menyimpan path pribadi dalam `.dvc/config.local` yang diabaikan Git. Untuk tim lintas komputer, gunakan remote bersama seperti object storage dan konfigurasi akses yang sesuai. Kredensial tidak ditaruh di notebook atau Git.

32. Praktik C - Training dan MLflow

Buka `configs/quick.yaml`. Ada dua kandidat `max_depth`, satu nilai `n_estimators`, dan dua fold. Ini menghasilkan empat fitting cross-validation, lalu fitting ulang model pemenang. Grid lengkap di `params.yaml` berisi 24 kandidat dan tiga fold.

Di **Terminal A**:

```
.\.venv\Scripts\python.exe -m house_price train --config configs/quick.yaml --output artifacts/classroom-01 --tracking-uri
http://127.0.0.1:5000
```

Hasil yang diharapkan: program mencetak `run_id`, `model_uri`, path model lokal, checksum dataset, metrik, dan parameter terbaik. Folder output berisi:

```
artifacts/classroom-01/
  model/           Pipeline MLflow yang dapat dimuat kembali
  metrics.json     MSE, RMSE, R2, dan CV MSE
  run.json         Identitas run, URI model, dan checksum dataset
```

Jangan mengharuskan angka metrik tertentu di kelas. Periksa bahwa data, seed, parameter, dan versi library sama sebelum membandingkan hasil.

Penjelasan pipeline

`StandardScaler -> RandomForestRegressor` disimpan sebagai satu objek. GridSearchCV melatih scaler hanya pada data training tiap fold. Test set tidak digunakan untuk memilih kandidat terbaik. Random Forest umumnya tidak membutuhkan scaling, tetapi tahap ini dipertahankan dari alur pembelajaran sebelumnya dan sekarang tersimpan bersama model.

MSE lebih kecil berarti rata-rata kesalahan kuadrat lebih kecil. RMSE berada pada satuan target. R^2 membandingkan kemampuan prediksi dengan baseline rata-rata dan dapat bernilai negatif. `cv_mse` adalah skor cross-validation pada data training, sedangkan `mse` adalah hasil pada test set. Jangan membandingkan keduanya seolah-olah dihitung pada sampel yang sama.

Buka MLflow UI

Server pada Terminal B sudah berjalan dari tahap persiapan. Buka http://127.0.0.1:5000. CLI di atas menggunakan `--tracking-uri http://127.0.0.1:5000`, sehingga run muncul pada server yang sama dengan notebook. CLI tanpa argumen tersebut memakai SQLite lokal sebagai mode offline; hasilnya berada pada store yang berbeda.

Minta peserta mencari:

1. Experiment `house-price-classroom`.
2. Parameter terbaik dan seed split.
3. MSE, RMSE, R^2, serta CV MSE.
4. Tag `dataset.sha256`.
5. Artifact `data_version/california_housing.csv.dvc`.
6. Model yang disimpan dan URI-nya.

Latihan diskusi: jika hanya nama registered model yang sama, apakah dua versi pasti memakai dataset yang sama? Bukti yang perlu diperiksa adalah tag dan artifact run, bukan nama model saja.

33. Praktik D - Load model dan prediksi

Di **Terminal A**, gunakan pipeline portabel dari run sebelumnya:

```
.\.venv\Scripts\python.exe -m house_price predict --model-uri artifacts/classroom-01/model --input
data/raw/california_housing.csv --output predictions-classroom-01.csv
Get-Content predictions-classroom-01.csv -TotalCount 6
```

Hasil yang diharapkan: CSV mempunyai kolom `prediction` dan 20.640 baris prediksi untuk baseline. Perintah ini merupakan demo inference pada seluruh dataset; evaluasi model tetap menggunakan test set dari training.

Untuk memuat melalui MLflow, ambil URI yang benar dari `run.json`:

```
$savedRun = Get-Content artifacts/classroom-01/run.json -Raw | ConvertFrom-Json
.\.venv\Scripts\python.exe -m house_price predict --model-uri $savedRun.model_uri --tracking-uri http://127.0.0.1:5000
--input data/raw/california_housing.csv --output predictions-from-mlflow.csv
```

Input harus mempunyai delapan fitur yang tercantum di README. Urutan kolom akan dinormalisasi. Kolom target boleh tetap ada dan tidak dipakai sebagai fitur. Jangan melakukan `scaler.fit_transform()` lagi saat inference; scaler di dalam pipeline sudah mempelajari statistik training.

Untuk praktik notebook, pilih kernel `.venv` dan jalankan cell berurutan. Bagian evaluasi memeriksa checksum dataset dan memakai pembagian data yang sama dengan training. Jika kernel di-restart, gunakan `run.json` untuk menemukan model; variabel Python lama tidak otomatis tersedia.

34. Praktik E - Tes dan pipeline DVC

Di **Terminal A**:

```
.\.venv\Scripts\python.exe -m ruff check .
.\.venv\Scripts\python.exe -m ruff format --check .
.\.venv\Scripts\python.exe -m pytest
.\.venv\Scripts\python.exe scripts/check_notebook.py
```

Jelaskan contoh perilaku yang diuji: dataset yang berubah tanpa metadata ditolak, restore DVC mengembalikan byte yang sama, preprocessing tersimpan bersama model, prediksi setelah load identik, dan output lama tidak ditimpa.

Tes menggunakan data sintetis kecil. Tes yang lolos membuktikan perilaku program sesuai pemeriksaan tersebut; kualitas model pada kebutuhan bisnis memerlukan evaluasi tambahan.

Menjalankan stage DVC

Untuk demo singkat, salin konfigurasi quick ke `params.yaml` pada branch latihan. Simpan perubahan tersebut sebagai bagian eksperimen kelas.

```
Copy-Item configs/quick.yaml params.yaml
$env:PATH = "$PWD\.venv\Scripts;$env:PATH"
$env:MLFLOW_TRACKING_URI = "http://127.0.0.1:5000"
.\.venv\Scripts\python.exe -m dvc repro
.\.venv\Scripts\python.exe -m dvc repro
```

PATH ditambahkan agar perintah `python` di stage memakai environment proyek. Run pertama menghasilkan model, metrik, laporan, dan `dvc.lock`. Run kedua seharusnya melewati stage yang inputnya tidak berubah.

Ubah satu nilai `max_depth` di `params.yaml`, lalu jalankan `dvc repro` kembali. DVC mendeteksi perubahan parameter dan menjalankan ulang stage. Output stage berada di `artifacts/training`; hasil demo CLI sebelumnya tetap berada di `artifacts/classroom-01`.

Commit `params.yaml` dan `dvc.lock` setelah hasil eksperimen dipilih. DVC menyimpan model portabel sebagai output stage. Riwayat UI dan registry MLflow memerlukan database/artifact store MLflow; pemulihan model DVC tidak otomatis memulihkan seluruh database tracking.

35. Praktik F - Docker

Di **Terminal C**, pastikan Docker Desktop aktif dengan engine Linux:

```
docker info
docker build -t house-price:classroom .
docker run --rm house-price:classroom --help
docker run --rm --network none --entrypoint python house-price:classroom /app/scripts/smoke_test.py
```

Hasil yang diharapkan: image berhasil dibangun; CLI tampil; smoke test melaporkan training, load, dan prediksi berhasil. `--network none` memastikan tes sintetis ini tidak bergantung pada unduhan atau server luar.

Tunjukkan Dockerfile: image dasar Python, instalasi paket, pengguna non-root, working directory `/workspace`, dan entrypoint CLI. `.dockerignore` membatasi build context ke kode serta file yang dibutuhkan. CSV, cache, model lokal, dan file rahasia tidak masuk image.

Gunakan model yang telah dibuat untuk prediksi dalam container:

```
docker run --rm --mount "type=bind,source=$($PWD.Path),target=/workspace" house-price:classroom predict --model-uri
artifacts/classroom-01/model --input data/raw/california_housing.csv --output predictions-container.csv
```

Mount menghubungkan folder host ke `/workspace`. File prediksi muncul kembali di folder host. Jika ingin training dalam container dengan konfigurasi kelas:

```
docker run --rm --mount "type=bind,source=$($PWD.Path),target=/workspace" house-price:classroom train --config
configs/quick.yaml --output artifacts/container-classroom-01
```

Untuk host Linux, sesuaikan izin folder mount dengan pengguna container, misalnya `--user "$(id -u):$(id -g)"`. Path model portabel dapat dipakai antar host/container. URI artifact lokal MLflow bisa menunjuk path khusus host; gunakan tracking server bersama untuk akses lintas mesin.

36. Praktik G - GitHub Actions dan GHCR

Baca workflow bersama peserta

Workflow ada di `.github/workflows/ci-cd.yml`.

Job	Isi	Kapan job berikutnya boleh berjalan?
`quality`	Instalasi, lint, format, tes Ubuntu/Windows, validasi notebook, build paket	Semua matrix berhasil
`container`	Build image dan smoke test tanpa jaringan	Container berhasil diuji
`publish`	Build kandidat, uji kandidat yang sama, login, lalu push GHCR	Hanya event/branch publikasi yang diizinkan

Pull request menjalankan CI dan pengujian container. Push ke `main`/`master` atau tag `v*` dapat mempublikasikan image setelah semua pemeriksaan lolos. `workflow_dispatch` menyediakan pemicu manual dari branch utama. Job publish mempunyai izin `packages: write`; job lain tidak membutuhkannya.

`GITHUB_TOKEN` disediakan GitHub saat workflow berjalan. Tidak perlu membuat PAT khusus untuk login workflow. Kebijakan organisasi atau package yang sudah ada mungkin memerlukan pemberian akses repository ke package.

Latihan pull request

Di fork pribadi, buat branch dan ubah satu komentar atau dokumentasi:

```
git switch -c latihan/ci
git add docs/LATIHAN_PESERTA.md
git commit -m "Add classroom learning notes"
git push -u origin latihan/ci
```

Pastikan file benar-benar diubah sebelum commit. Buat pull request menuju `main` pada fork sendiri, lalu buka tab Actions. Tunjukkan log satu langkah, hasil tes, dan artifact laporan. Jika Actions pada fork belum aktif, aktifkan melalui tab Actions terlebih dahulu.

Pemilik fork dapat melakukan merge setelah CI lolos. Push hasil merge ke branch utama akan memicu job publish. Jangan mengirim latihan semua peserta langsung ke branch utama repository pengajar.

Verifikasi hasil CD

Untuk repository pengajar, nama image adalah:

```
ghcr.io/bayuzen19/mlops_house_price
```

Untuk fork, owner mengikuti pemilik fork. Workflow membuat tag SHA commit, tag branch atau tag Git, dan `latest` hanya pada default branch. Tag SHA membantu memilih image dari commit tertentu.

Verifikasi tiga hal: job publish berhasil, package/version muncul di GHCR, dan image dapat di-pull serta dijalankan. Contoh untuk image pengajar setelah publikasi berhasil:

```
docker pull ghcr.io/bayuzen19/mlops_house_price:latest
docker run --rm ghcr.io/bayuzen19/mlops_house_price:latest --help
```

Package private memerlukan login yang memiliki akses baca package. Untuk kelas publik, pemilik package dapat mengatur visibilitas package sesuai kebutuhan. Tidak perlu membagikan token pengajar kepada peserta.

37. Demonstrasi kegagalan yang terkontrol

Lakukan latihan kegagalan pada branch/folder latihan. Setelah itu, kembalikan perubahan yang disengaja.

Percobaan	Respons yang diharapkan	Konsep
Ubah CSV tanpa `dvc add`	Training menolak checksum yang berbeda	Data lineage
Hapus satu fitur dari input prediksi	Pesan kolom fitur tidak lengkap	Kontrak input
Gunakan file output prediksi yang sudah ada	Perintah menolak overwrite	Perlindungan hasil
Tambahkan unused import pada branch latihan	Ruff/CI gagal	Pemeriksaan otomatis
Ubah parameter stage lalu `dvc repro`	Training diulang	Dependency tracking
Jalankan ulang `dvc repro` tanpa perubahan	Stage dilewati	Reproducibility dan cache

Minta peserta membaca pesan error pertama yang relevan, menentukan tahap yang gagal, lalu memperbaiki penyebabnya. Hindari mengulang semua langkah instalasi untuk setiap error.

38. Troubleshooting

Gejala	Pemeriksaan dan tindakan
`No module named house_price`	Jalankan instalasi dari root repo menggunakan `.venv\Scripts\python.exe -m pip install -r requirements-dev.txt`; periksa kernel notebook
Python bukan 3.13	Buat environment dengan `py -3.13 -m venv .venv` menggunakan instalasi Python yang sesuai
`Git not found`	Instal Git, buka ulang terminal/editor, lalu uji `git --version`
CSV hilang tetapi file `.dvc` ada	Dataset awal: `prepare-data --from-source`; versi modifikasi: `dvc checkout`/`dvc pull` dari cache/remote yang benar
Bootstrap menolak checksum	Versi sumber berbeda dari metadata. Pulihkan versi yang benar; jangan mengganti metadata hanya agar lolos
`CSV berbeda dari versi DVC`	Tinjau perubahan; jalankan `dvc add` untuk versi baru yang disengaja atau restore versi sebelumnya
Folder output tidak kosong	Pilih folder output baru. Untuk stage DVC, jalankan melalui `dvc repro` agar output stage dikelola DVC
`UntrustedTypesFoundException` untuk Tree	Gunakan fungsi training proyek yang menyertakan daftar tipe `skops` spesifik untuk model yang dilatih sendiri
MLflow port 5000 dipakai	Periksa apakah proses yang ada sudah server MLflow kelas. Jangan menyalakan UI kedua atau mematikan layanan yang belum dikenali. Jika sengaja memakai port alternatif, ubah URI notebook/CLI agar sama dengan server
Model URI tidak ditemukan	Gunakan tracking URI dari run yang sama; cek `run.json`, atau gunakan folder model portabel
DVC menganggap snapshot MLflow sebagai stage	Pastikan `.mlflow/`, `mlruns/`, dan `mlartifacts/` ada di `.dvcignore`
`WinError 3` pada path cache yang panjang	Gunakan lokasi repo/cache lebih pendek; run cache DVC menambahkan hash panjang pada path
Docker CLI ada tetapi Server tidak merespons	Buka Docker Desktop dan baca error startup. Instalasi CLI saja belum berarti engine aktif
Docker error pada `dockerInference`	Ini error startup Docker Desktop yang ditemui pada mesin pengajar; perbaiki runtime Docker sebelum build. Jangan memilih factory reset sebagai langkah pertama karena bisa menghapus data Docker
GHCR `permission_denied`	Periksa `packages: write`, kebijakan Actions organisasi, dan akses repository pada package
Job publish dilewati	Periksa event dan branch. Pull request memang hanya menjalankan pemeriksaan
Image sudah dipush tetapi peserta gagal pull	Periksa owner/tag dan visibilitas package; package private memerlukan akses

39. Penilaian dan bahan diskusi

Minta peserta menyerahkan bukti, bukan hanya pernyataan “sudah jalan”.

Komponen	Bobot	Bukti
Environment dan pemahaman struktur	10	CLI berjalan dan penjelasan peran modul
DVC	25	Metadata dua versi dan checksum setelah restore
MLflow	25	Run, konfigurasi, tag dataset, metrik, dan URI model
Inference	15	CSV hasil prediksi dan penjelasan preprocessing
CI/CD	25	URL workflow, status pemeriksaan, serta tag image GHCR

Jika Docker pada laptop peserta tidak tersedia, peserta tetap dapat mengerjakan latihan Python/DVC/MLflow dan mengamati build Docker di GitHub-hosted runner. Catat bagian yang belum dijalankan lokal.

Pertanyaan penutup:

1. Apa yang tidak tersimpan jika hanya commit notebook ke Git?
2. Mengapa `dvc add` masih perlu diikuti commit metadata?
3. Mengapa scaler harus ikut disimpan?
4. Apa perbedaan “best estimator pada satu grid search” dan “model terbaik untuk dipakai bisnis”?
5. Mengapa CI memakai dataset kecil dan tidak menjalankan grid penuh setiap push?
6. Apakah image di GHCR otomatis berarti aplikasi sudah berjalan di server?
7. Bukti apa yang diperlukan untuk mengulang eksperimen minggu lalu?

40. Referensi resmi

- DVC: mulai menggunakan version control data
- DVC add dan checkout
- MLflow: logging dan loading model sklearn
- MLflow 3: URI model dari hasil logging
- GitHub Actions: build dan test Python
- GitHub Actions: publish Docker image

Lembar kerja yang dapat dibagikan langsung kepada peserta tersedia di LATIHAN_PESERTA.md.